



Stand: 21.05.2026

Zentrales Steuerungs- und Loganalysemodul für Shopsysteme

Firma: 4SELLERS

Zeitaufwand: 80h

Erstellt von: Benjamin Biber

Inhaltsverzeichnis

1. Projektbeschreibung.....	4
1.1. Einleitung.....	4
1.2. Projektumfeld	4
1.2.1. Die 4SELLERS GmbH	4
1.2.2. 4SELLERS Shopsystem	4
1.2.3. Shop-Toolbox	4
1.3. Ist-Analyse	5
1.4. Soll-Konzept.....	5
1.4.1. Log-Analyse	5
1.4.2. IIS-Steuerung	5
1.4.3. Einstellungsdienst	6
2. Aufwand und Nutzen	6
2.1. Der Nutzen für die Firma 4SELLERS GmbH	6
2.2. Kosten	6
2.3. Kostendeckung der Entwicklung	6
2.4. Nutzen für den Anwender.....	7
3. Projektplanung.....	7
3.1. Projektphasen	7
3.2. Technologien.....	7
3.2.1. Auswahl.....	7
3.2.2. Vergleich der Datenbankanbindung	8
3.2.3. Vergleich der Windows-Ereignisprotokoll-APIs	9
3.2.4. Auswahl des Speicherorts der Datenbankdatei	10
3.3. Ressourcenplanung.....	10
4. Projektumsetzung	11
4.1. Architektur & Projektstruktur	11
4.2. Implementierung der Log-Funktion	11
4.2.1. Aufbau einer Logmeldung	11
4.2.2. Log-Service	12
4.2.3. Anzeige der Logs.....	12
4.3. Implementieren eines IIS-Services.....	13

4.3.1.	IIS-Service.....	13
4.3.2.	Erweiterter Neustart	13
4.3.3.	Bedienung der IIS-Funktionen.....	1
4.4.	Implementieren eines Settings-Services	1
4.4.1.	Settings-Service	2
4.4.2.	Einstellungen speichern.....	2
4.4.3.	Einstellungsseite	2
4.4.4.	ToolboxSettings	2
5.	Testen.....	3
5.1.	Designänderungen	3
5.1.1.	Anzeige der Logmeldungen.....	4
5.1.2.	Verbessern der Ansicht für kleinere Bildschirme	4
6.	Fazit und Weiterentwicklung	4
6.1.	Fazit	4
6.2.	Weiterentwicklung.....	4
6.2.1.	Verwaltung von Staging VMs	4
6.2.2.	Verwaltung von Shopsystem Extensions.....	5
6.2.3.	Verwaltung von Shopsystem Themes	5
7.	Glossar.....	6
8.	Quellen.....	9
9.	Anhänge	10
9.1.	Aufwand und Nutzen	10
9.1.1.	Bearbeitungszeiten im Vorher-Nachher-Vergleich (in Sekunden)	10
9.1.2.	Kostendeckung.....	10
10.	Projekturnsetzung.....	11
10.1.1.	Startseite	11
10.1.2.	Log-Suche.....	12
10.1.3.	Erweiterter Neustart	13
10.1.4.	Log-Popover	13
10.1.5.	Datenbankstruktur.....	14

1. Projektbeschreibung

1.1. Einleitung

Das Projekt "Zentrales Steuerungs- und Loganalysemodul für Shopsysteme" wird in dieser Dokumentation in seinem Umfang, seinem Ablauf und seiner Umsetzung dargestellt. Es wurde als Abschlussprojekt für die Ausbildung als Fachinformatiker in der Fachrichtung Anwendungsentwicklung durchgeführt.

Im Folgenden wird thematisiert, in welchem System das Projekt umgesetzt wurde. Außerdem wird erläutert, warum das Projekt umgesetzt werden sollte und welche Teilaspekte dabei berücksichtigt wurden.

1.2. Projektumfeld

1.2.1. Die 4SELLERS GmbH

Die 4SELLERS GmbH ist ein auf E-Commerce spezialisiertes Softwareunternehmen mit Hauptsitz in Rain am Lech und weiteren Standorten in Aachen, Leipzig sowie der Tochtergesellschaft CRM Solutions in Hamburg.

Das zentrale Produktportfolio unter dem Namen „4SELLERS“ umfasst eine Vielzahl an Softwarelösungen für den Onlinehandel. Dazu gehören insbesondere ein vollständig integriertes ERP-System auf Basis von Sage 100 sowie ein eigenes Shopsystem. Die Lösungen ermöglichen es Händlern, ihre Produkte effizient über verschiedene Kanäle – wie z. B. den eigenen Online-Shop, Amazon oder eBay – zu vertreiben.

Durch die Automatisierung zentraler Prozesse können Kunden Zeit und Ressourcen sparen und ihre Reichweite erhöhen. Mit über 300 Kunden und rund 200 Mitarbeitenden zählt 4SELLERS zu den führenden deutschen Anbietern von E-Commerce-Komplettlösungen für kleine und mittelständische Unternehmen.

1.2.2. 4SELLERS Shopsystem

Das 4SELLERS Shopsystem verbindet die SAGE 100 mit einem vollfunktionalen Online-Shop. Technologisch ist das Shopsystem in der Programmiersprache C# auf Basis von .NET 10 umgesetzt und wird über die Internet Information Services (IIS) von Microsoft als Webanwendung auf Windows-Systemen gehostet.

Zur Unterstützung des täglichen Entwicklungs-Workflows und zur Administration dieser Instanzen dient die interne Shop-Toolbox, in welche die im Rahmen dieses Projekts entwickelten Module integriert werden.

1.2.3. Shop-Toolbox

Die Shop-Toolbox ist ein zentrales, internes Werkzeug, welches im Produktteam Shopsystem zur Unterstützung der täglichen Entwicklungsarbeit eingesetzt wird. Sie bündelt administrative Funktionen in einer einheitlichen Oberfläche, um die Verwaltung der verschiedenen Shop-Instanzen zu vereinfachen. Die Anwendung ist technologisch als Blazor Desktop Anwendung (WPF-Host mit WebView2) umgesetzt, welche es ermöglicht Desktop-Anwendungen mithilfe von HTML, CSS & C# zu schreiben. Das vorliegende Projekt erweitert diese Toolbox um spezialisierte Module für die IIS-Steuerung und die Loganalyse, um Medienbrüche zwischen verschiedenen Systemwerkzeugen zu eliminieren

1.3. Ist-Analyse

Im aktuellen Entwicklungsprozess werden administrative Aufgaben manuell ausgeführt, wodurch die Effizienz der Entwickler reduziert wird. Zur Analyse von Fehlermeldungen muss die Windows-Ereignisanzeige separat geöffnet und der jeweilige Eintrag manuell ermittelt werden, da keine integrierte Such- oder Filterfunktion zur Verfügung steht.

Der Neustart einer Shop-Instanz erfordert mehrere aufeinanderfolgende Schritte: Zunächst wird der entsprechende Dienst im IIS-Manager manuell gestoppt. Anschließend ist abzuwarten, bis die CSS-Caches vollständig geleert sind, bevor der CSS-Bundler manuell entfernt werden kann. Erst danach wird der Dienst neu gestartet. Da diese Schritte in einer festen Reihenfolge ausgeführt werden müssen, ist der Prozess sowohl zeitintensiv als auch fehleranfällig.

Der daraus resultierende Medienbruch zwischen Shop-Toolbox, IIS-Manager und Windows-Ereignisanzeige verursacht einen erhöhten Zeitaufwand und reduziert die Arbeitseffizienz im täglichen Entwicklungsbetrieb.

1.4. Soll-Konzept

Ziel des Projekts ist die funktionale Erweiterung der bestehenden Shop-Toolbox um zwei zentrale Module, damit die Anzahl der parallel verwendeten Anwendungen im Entwicklungsprozess reduziert und somit zeitintensive Medienbrüche vermieden werden. Durch die Zentralisierung administrativer Aufgaben wird der manuelle Aufwand minimiert, wodurch komplexe Abläufe im Shopsystem mit einer optimierten Anzahl an User-Interaktionen effizienter durchgeführt werden können.

Für die im Folgenden beschriebenen Module gelten folgende Akzeptanzkriterien:

- Vollständige und chronologisch korrekte Anzeige aller 4SELLERS-Logeinträge aus der Windows-Ereignisanzeige innerhalb der Shop-Toolbox, ohne dass die Ereignisanzeige separat geöffnet werden muss.
- Garantierte Leerung sämtlicher relevanter Caches nach einem erweiterten Neustart über die Shop-Toolbox.
- Dauerhafte Persistenz aller Nutzereinstellungen über Updates und Neuinstallationen hinweg.

- Erweiterbarkeit des Einstellungs-Datenmodells um neue Felder ohne größeren Aufwand.

1.4.1. Log-Analyse

Die Shop-Toolbox soll um eine Funktion zur automatisierten Verarbeitung und Darstellung von Systemlogs erweitert werden. Die vom Shopsystem 4SELLERS generierten Logdaten sollen dabei automatisch ausgelesen und dem Nutzer in einer strukturierten, tabellarischen Übersicht zugänglich gemacht werden.

Um die Auswertung der Logdaten effizient zu gestalten, soll die Oberfläche zusätzlich Funktionen zur Durchsuchung, Filterung und Sortierung der angezeigten Einträge bereitstellen. Dadurch kann der Nutzer relevante Informationen gezielt und ohne manuellen Mehraufwand identifizieren.

1.4.2. IIS-Steuerung

Die Shop-Toolbox soll um eine direkte Steuerungsmöglichkeit für die im Internet Information Services (IIS) gehosteten Shopsysteme erweitert werden. Hierfür sollen Schaltflächen zum Starten, Stoppen und Neustarten des entsprechenden IIS-Dienstes in die Oberfläche integriert werden, um administrative Eingriffe ohne den Umweg über den Server selbst durchführen zu können.

Um einen vollständigen und konsistenten Neustart des Shopsystems sicherzustellen, soll der Neustartvorgang zusätzlich eine automatisierte Leerung des CSS-Caches sowie eine Zurücksetzung des CSS-Bundlers umfassen. Dadurch wird gewährleistet, dass nach dem Neustart keine veralteten Stilinformationen im System verbleiben und der Shop in einem definierten, sauberen Zustand hochfährt.

1.4.3. Einstellungsdienst

Um eine möglichst reibungslose und individuell angepasste Nutzung der Shop-Toolbox zu gewährleisten, ist zudem die Implementierung eines Einstellungsdienstes vorgesehen. Über eine dedizierte Einstellungsseite sollen Nutzer die Konfiguration der Log-Analyse sowie der IIS-Steuerung benutzerbezogen anpassen können.

2. Aufwand und Nutzen

2.1. Der Nutzen für die Firma 4SELLERS GmbH

Der 4SELLERS GmbH entstehen durch die Entwicklung des Moduls Kosten, welche jedoch durch die signifikant reduzierten Aufwände bei der Fehlersuche und Systemadministration amortisiert werden. Darüber hinaus kann das hier angewandte Automatisierungskonzept als Grundlage dienen, um weitere administrative Teilbereiche des Shopsystems zu optimieren und somit eine durchgehend höhere Qualität und Zeitersparnis zu erzielen.

2.2. Kosten

Am Anfang wurden die Kosten, die durch das Projekt anfallen, kalkuliert. Bei der Berechnung der Personalkosten dürfen die tatsächlichen Stundensätze nicht herangezogen werden, da diese der Vertraulichkeit unterliegen. Stattdessen wird an dieser Stelle mit fiktiven Werten gearbeitet. Der Stundensatz eines Auszubildenden beläuft sich auf 6 Euro, der eines Mitarbeiters auf 35 Euro.

Die Gemeinkosten wie Lizenz-, Strom-, Heizkosten etc. belaufen sich auf 10 Euro pro Stunde.

Der nachfolgenden Tabelle kann die Aufteilung der Projektkosten entnommen werden.

Art	Einzelkosten in €	Anzahl in Stunden	Höhe in €
Lohnkosten (Auszubildende)	6	80	480
Gemeinkosten	10	80	800
Gesamt			1280

2.3. Kostendeckung der Entwicklung

Die durchgeführten Zeitmessungen belegen eine durchschnittliche Ersparnis von ca. 20 Sekunden pro Vorgang ([siehe 9.1.1 Bearbeitungszeiten im Vorher-Nachher-Vergleich \(in Sekunden\)](#)). Bei einer geschätzten Nutzungsfrequenz von 15 bis 20 Vorgängen täglich über fünf Nutzer ergibt sich eine kumulierte Zeitersparnis von rund 29 Minuten pro Arbeitstag, was bei einem Stundenlohn von 35€/Stunde einem monetären Gegenwert von ca. 17 € täglich entspricht. Bei Gesamtentwicklungskosten von 1.280 € errechnet sich daraus ein Break-Even-Punkt nach circa 75 Arbeitstagen (siehe [9.1.2 Kostendeckung](#)), entsprechend einem Zeitraum von rund 3,5 Monaten nach Inbetriebnahme. Durch den Einsatz der Shop-Toolbox entfällt für die Nutzer der bislang erforderliche Wechsel zwischen verschiedenen Systemwerkzeugen, was sowohl den damit verbundenen Medienbruch als auch den resultierenden Zeitaufwand signifikant reduziert. Die eingesparte Zeit kann folglich für wertschöpfende Tätigkeiten im Entwicklungsprozess genutzt werden, wodurch sich die Investition in die Shop-Toolbox mittel- bis langfristig amortisiert.

2.4. Nutzen für den Anwender

Die Shop-Toolbox bietet dem Anwender einen unmittelbaren Mehrwert in zwei zentralen Bereichen. Zum einen beschleunigt die integrierte Such- und Filterfunktion der Systemlogs die Fehlerdiagnose, zum anderen erlaubt die vereinfachte Neustartfunktion ein schnelles und zuverlässiges Testen des Shopsystems – ohne den Umweg über externe Werkzeuge.

3. Projektplanung

Der folgende Abschnitt handelt von der Planung des Projekts. Es wird erläutert, welche Ressourcen für die Projektumsetzung nötig waren und wie diese eingesetzt wurden. Zudem wird dargestellt, wie bei der Technologieauswahl verschiedene Nutzwertanalysen angewandt wurden, um die jeweils geeignetste Lösung zu ermitteln.

3.1. Projektphasen

Die projektierte Gesamtdauer von 80 Stunden konnte vollständig eingehalten werden. Im Vergleich zum ursprünglichen Projektantrag ergaben sich jedoch geringfügige Abweichungen in der phasenspezifischen Zeitplanung. In der Phase "Analyse und Konzeptionierung" wurde ein Zeitaufwand von zwei Stunden weniger als geplant benötigt. Das dadurch entstandene Zeitbudget wurde im Rahmen der Dokumentationsphase vollständig aufgebraucht, sodass die Gesamtstundenzahl unverändert blieb.

Die nachfolgende Tabelle stellt die geplanten Zeiten den tatsächlich aufgewendeten Zeiten phasenweise gegenüber.

Projektphase	Geplante Zeit in Stunden	Benötigte Zeit in Stunden
Analyse und Konzeptionierung	12 h	10 h
Implementierung	40 h	40 h
Test	18 h	18 h
Dokumentation	10 h	12 h
Gesamt	80 h	80 h

3.2. Technologien

3.2.1. Auswahl

Bei der Auswahl der eingesetzten Technologien wurde darauf geachtet, dass sich die Umsetzung des neuen Features möglichst an der bestehenden Codebasis der Shop-Toolbox orientiert, um eine konsistente und wartbare Erweiterung der Anwendung zu gewährleisten. Als Programmiersprachen kamen entsprechend die bereits im Projekt etablierten Sprachen C#, HTML und CSS zum Einsatz.

Für die zentralen Technologieentscheidungen – die Datenbankbindung (Kapitel 3.1.2) sowie die Auswahl der Windows-Ereignisprotokoll-API (Kapitel 3.1.3), wurde jeweils eine Nutzwertanalyse durchgeführt. Die Kriterien wurden aus den fachlichen und technischen Anforderungen der Shop-Toolbox abgeleitet: Einrichtungs- und Entwicklungsaufwand spiegeln den Implementierungs- und Einarbeitungsaufwand wider, Wartbarkeit berücksichtigt die langfristige Pflegebarkeit innerhalb der bestehenden Codebasis, Performance die Laufzeitanforderungen, und externe Abhängigkeiten bewerten das Risiko zusätzlicher Komponenten außerhalb der Anwendung.

Die Gewichtung der Kriterien orientiert sich am konkreten Anwendungsfall: Aspekte mit großem Einfluss auf den langfristigen Betrieb (Wartbarkeit, Einrichtungsaufwand) wurden höher gewichtet als Kriterien, die im vorliegenden Szenario eine untergeordnete Rolle spielen (z. B. Performance bei kleinen Konfigurationsdaten). Die Bewertung jedes Kriteriums erfolgt auf einer Skala von 1 bis 5: 1 bedeutet „sehr ungeeignet / hoher Aufwand bzw. hohes Risiko“, 2 „eher ungeeignet / erhöhter Aufwand bzw. erhöhtes Risiko“, 3 „neutral / akzeptabel“, 4 „gut geeignet / geringer Aufwand bzw. geringes Risiko“ und 5 „sehr gut geeignet / minimaler Aufwand bzw. minimales Risiko“. Die Einstufung je Kriterium wurde anhand der Praxiserfahrung im bestehenden Projekt, der offiziellen Dokumentation der jeweiligen Technologien sowie Tests im Rahmen eines Prototyps vorgenommen.

Der Gesamtnutzwert ergibt sich aus der Summe der mit der jeweiligen Gewichtung multiplizierten Einzelbewertungen; die Option mit dem höchsten Gesamtwert wurde für die Umsetzung ausgewählt.

Die Entscheidung, keine neuen Technologien einzuführen, minimiert den Einarbeitungsaufwand und reduziert das Risiko von Kompatibilitätsproblemen. Ein Wechsel auf ein alternatives UI-Framework oder eine andere Programmiersprache hätte die Integration in die bestehende Blazor-Architektur erheblich erschwert.

Die Verwaltung des Quellcodes erfolgt über das bereits vorhandene Git-Repository, das eine strukturierte Versionsverwaltung und die vollständige Nachvollziehbarkeit aller Änderungen sicherstellt.

Für die persistente Speicherung der benutzerspezifischen Einstellungen wurde SQLite in Kombination mit dem Entity Framework Core eingesetzt. SQLite ermöglicht dabei eine lokale, dateibasierte Datenspeicherung ohne den Aufwand eines separaten Datenbankservers, während Entity Framework Core als objektrelationaler Mapper den Datenbankzugriff abstrahiert und den Implementierungsaufwand auf ein Minimum reduziert. Ein potenzielles Risiko dieser Entscheidung liegt in der eingeschränkten Skalierbarkeit von SQLite bei sehr großen Datenmengen, da es sich jedoch ausschließlich um benutzerspezifische Konfigurationseinstellungen handelt, ist dieses Risiko für den vorliegenden Anwendungsfall als vernachlässigbar einzustufen.

Für die Gestaltung des User-Interfaces wurde die in der Shop-Toolbox bereits eingesetzte MudBlazor-Komponentenbibliothek in Kombination mit dem CSS-Framework Bootstrap verwendet. Während MudBlazor eine einheitliche Komponentenbasis und ein modernes Erscheinungsbild innerhalb der bestehenden Anwendung sicherstellt, ermöglicht Bootstrap eine flexible und strukturierte Layoutgestaltung der Benutzeroberfläche. Das Risiko einer externen Abhängigkeit von der MudBlazor-Bibliothek wurde bewusst in Kauf genommen, da sie bereits produktiv eingesetzt wird und somit keine neue Abhängigkeit darstellt.

3.2.2. Vergleich der Datenbankanbindung

Im Rahmen der Technologieauswahl wurde geprüft, welcher Ansatz zur Datenbankanbindung den Anforderungen der Shop-Toolbox am besten entspricht. Die Bewertung erfolgte anhand der Kriterien Einrichtungsaufwand, Wartbarkeit und Eignung für den vorliegenden Anwendungsfall.

Kriterium	Gewichtung	EF Core + SQLite	Manuelles SQL + ext. DB-Server
Einrichtungsaufwand	25 %	5	1
Entwicklungsaufwand	20 %	5	2
Wartbarkeit	25 %	5	3
Performanz	15 %	4	5
Externe Abhängigkeiten	15 %	5	1
Gesamt	100 %	4,85	2,30

Da es sich bei den zu speichernden Daten ausschließlich um benutzerspezifische Konfigurationseinstellungen handelt und kein externer Datenbankserver erforderlich sein soll, überwiegen die Vorteile von EF Core mit SQLite gegenüber einem manuellen SQL-Ansatz deutlich. Ein weiterer entscheidender Vorteil liegt in der Wartbarkeit: Änderungen an der Datenbankstruktur werden über EF Core Migrationen verwaltet und gemeinsam mit dem Anwendungscode versioniert (siehe [10.1.5_Datenbankstruktur](#)

). Dadurch wird sichergestellt, dass Datenbankschema und Clientcode stets synchron sind und Anpassungen ausschließlich im Rahmen eines regulären Anwendungsupdates eingespielt werden.

Dieser Aspekt ist besonders relevant, da der Einstellungsdienst explizit darauf ausgelegt ist, zukünftig um weitere Konfigurationsfelder ergänzt zu werden. Mit einem manuellen SQL-Ansatz würde jede solche Erweiterung eine separate Datenbankmigration erfordern, die fehleranfällig ist und nicht automatisch mit dem Anwendungsupdate ausgeliefert werden kann. EF Core löst dieses Problem, indem Migrationen als Teil des Quellcodes versioniert und beim Anwendungsstart automatisch angewendet werden.

3.2.3. Vergleich der Windows-Ereignisprotokoll-APIs

Für das Auslesen der Windows-Ereignisprotokolle stehen in .NET grundsätzlich zwei APIs zur Verfügung: die modernere EventLogReader-API sowie die ältere EventLog-API. Da die Wahl der API direkten Einfluss auf Performance und Zuverlässigkeit der Log-Funktion hat, wurden beide Ansätze systematisch gegenübergestellt.

Kriterium	Gewichtung	EventLogReader	EventLog
Filterleistung	30 %	5	2
Performance	30 %	5	2
Inkrementelles Laden	20 %	5	1
Fehleranfälligkeit	10 %	5	2
Kompatibilität	10 %	4	5
Gesamt	100 %	4,90	2,10

Die Gegenüberstellung zeigt, dass die EventLogReader-API in allen relevanten Kriterien überlegen ist und daher als primäre Lesemethode eingesetzt wird. Besonders bei großen Log-Mengen, wie sie im laufenden Entwicklungsbetrieb eines Shopsystems regelmäßig anfallen, ist die XPath-basierte Filterung direkt auf Systemebene ein entscheidender Vorteil gegenüber dem nachträglichen Filtern im Code.

Da jedoch nicht auf allen Zielsystemen garantiert werden kann, dass diese API fehlerfrei zur Verfügung steht – etwa aufgrund eingeschränkter Berechtigungen oder abweichender Systemkonfigurationen – wird die ältere EventLog-API als automatischer Fallback implementiert. Dadurch wird sichergestellt, dass die Log-Funktion auch unter ungünstigen Systembedingungen zuverlässig funktioniert, ohne dass der Nutzer manuell eingreifen muss. Unter ungünstigen Systembedingungen sind dabei insbesondere fehlende Berechtigungen des ausführenden Benutzers auf das Windows-Ereignisprotokoll, ein deaktivierter oder nicht erreichbarer Windows-Ereignisprotokoll-Dienst, beschädigte oder gesperrte Logdateien sowie ältere Windows-Versionen ohne vollständige Unterstützung der EventLogReader-API zu verstehen. Dieses Fallback-Konzept folgt dem Prinzip der defensiven Programmierung und erhöht die Robustheit der Anwendung nachhaltig.

3.2.4. Auswahl des Speicherorts der Datenbankdatei

Neben der Wahl der Datenbankanbindung musste auch der geeignete Speicherort für die SQLite-Datenbankdatei bestimmt werden. Da die Shop-Toolbox regelmäßig aktualisiert wird, war es besonders wichtig, einen Speicherort zu wählen, der benutzerspezifische Einstellungen über Updates hinweg vollständig erhält.

Kriterium	Gewichtung	App-Verzeichnis	AppData Roaming
-----------	------------	-----------------	-----------------

Einstellungen bei Update erhalten	35 %	1	5
Benutzerspezifität	25 %	1	5
Schreibrechte	25 %	2	5
Synchronisierbarkeit	15 %	3	4
Gesamt	100 %	1,55	4,85

Da die Shop-Toolbox regelmäßig aktualisiert wird, wurde der AppData-Ordner als Speicherort der Datenbankdatei gewählt. Dadurch wird sichergestellt, dass benutzerspezifische Einstellungen bei einem Update vollständig erhalten bleiben und nicht erneut konfiguriert werden müssen.

Ein weiterer relevanter Aspekt ist die Benutzerspezifität: Da die Shop-Toolbox von mehreren Entwicklern auf gemeinsam genutzten Systemen eingesetzt werden kann, ist es wichtig, dass jeder Nutzer seine eigene, unabhängige Konfiguration pflegen kann. Eine Speicherung im App-Verzeichnis würde hingegen dazu führen, dass alle Nutzer dieselbe Konfigurationsdatei teilen, was bei unterschiedlichen Präferenzen oder Zuständigkeiten zu Konflikten führen könnte.

Darüber hinaus erfordert das App-Verzeichnis unter Umständen erhöhte Schreibrechte, was auf manchen Systemen zu Problemen führen kann. Der AppData-Ordner hingegen ist für jeden Nutzer ohne administrative Berechtigungen beschreibbar, was die Anwendung robuster und unkomplizierter im Einsatz macht.

3.3. Ressourcenplanung

Für die Durchführung des Projekts werden sowohl hardware- als auch softwareseitige Ressourcen benötigt. Als Arbeitsgrundlage dient eine Entwicklungsworkstation, auf der die integrierte Entwicklungsumgebung „JetBrains Rider 2025“ sowie der Microsoft Webserver „Internetinformationsdienste“ (IIS) installiert sind.

Darüber hinaus muss lokal eine lauffähige Instanz eines 4SELLERS-Standardshops in der aktuellen Version eingerichtet sein, da die entwickelte Erweiterung auf Basis dieser Umgebung implementiert und getestet wird. Diese Anforderung ergibt sich daraus, dass zentrale Funktionen der Erweiterung, insbesondere die IIS-Steuerung und die Log-Analyse, nur in Verbindung mit einer tatsächlich laufenden Shop-Instanz vollständig getestet werden können. Eine rein isolierte Testumgebung ohne laufenden Shop hätte keine aussagekräftigen Testergebnisse geliefert.

Alle eingesetzten Softwarekomponenten, darunter JetBrains Rider, der IIS sowie die verwendeten NuGet-Pakete, standen im Rahmen der Ausbildung zur Verfügung und verursachten keine zusätzlichen Lizenzkosten, die über die bereits in der Kostenkalkulation berücksichtigten Gemeinkosten hinausgehen. Externe Hardware oder zusätzliche Dienste waren für die Durchführung des Projekts nicht erforderlich.

4. Projektumsetzung

In diesem Abschnitt wird der gesamte Ablauf der Projektdurchführung dargestellt. Da es sich um eine Erweiterung der bestehenden Shop-Toolbox handelt, mussten keine neuen Projekte angelegt werden. Alle Funktionen wurden in die bereits vorhandene Blazor-Anwendung integriert. Die folgenden Features wurden im Rahmen des Projekts implementiert:

- Erstellung der Log-Anzeige mit integrierter Such-, Filter- und Sortierfunktion zur strukturierten Darstellung der 4SELLERS-Systemlogs
- Implementierung der IIS-Steuerung mit Schaltflächen zum Starten, Stoppen und Neustarten des gehosteten Shopsystems
- Entwicklung des erweiterten Neustarts, der neben dem IIS-Dienst automatisch den CSS-Cache leert und den CSS-Bundler zurücksetzt, um einen konsistenten Systemzustand zu gewährleisten
- Implementierung des Einstellungsdienstes mit persistenter, benutzerspezifischer Datenspeicherung auf Basis von EF Core und SQLite

4.1. Architektur & Projektstruktur

Die Implementierung verteilt sich auf zwei bestehende Projekte der Shop-Toolbox. Das Projekt „Toolbox“ enthält die Blazor-Komponenten und die Benutzeroberfläche, während das Projekt „Toolbox.Data“ die Datenmodelle und Services bereitstellt.

4.2. Implementierung der Log-Funktion

Die Log-Funktion wurde als zentrale Komponente der Startseite der Shop-Toolbox implementiert. Sie ermöglicht das automatische Auslesen der 4SELLERS-Einträge aus dem Windows-Ereignisprotokoll sowie deren strukturierte Darstellung und gezielte Durchsuchung innerhalb einer MudBlazor-Tabellenkomponente (siehe [9.2.1 Startseite](#)).

4.2.1. Aufbau einer Logmeldung

Zur strukturierten Verarbeitung der Log-Daten wurden mehrere Modellklassen implementiert. Die Klasse LogEntry repräsentiert einen einzelnen Log-Eintrag mit Zeitstempel, Schweregrad und Nachrichtentext. Identische aufeinanderfolgende Einträge werden dabei über das Count-Attribut gebündelt dargestellt, um die Übersichtlichkeit bei sich wiederholenden Meldungen zu erhöhen. Die Klasse LogMessage repräsentiert die geparte Struktur eines Eintrags und zerlegt den Rohtext in die Felder Origin, Metadata und Message, die dem im Shopsystem definierten Log-Format entsprechen. Der Schweregrad eines Eintrags wird über das Enum LogLevel abgebildet, das die Stufen Information, Warning, Error und Critical unterscheidet. Ergänzend kennzeichnet das Enum MessageType mit den Werten Default, ParseError und Loaded den Verarbeitungsstatus eines Eintrags und ermöglicht es, fehlerhaft geparte oder rein informative Systemeinträge in der

Oberfläche gesondert zu behandeln. Für die Übergabe der Ergebnisse aus dem Log-Service an die Startseite wurde zusätzlich die Klasse LogBatch eingeführt, die neben der Liste der gelesenen Einträge auch die zuletzt verarbeitete Record-ID je Log-Quelle als Dictionary mit sich führt. Diese Trennung zwischen Nutzdaten und Cursor-Informationen vereinfacht das inkrementelle Nachladen, da der Aufrufer den Cursor nach jedem Aufruf einfach übernehmen und beim nächsten Aufruf wieder mitgeben kann, ohne sich um die Detaillogik des Service kümmern zu müssen. Über das Enum ReloadOption wird darüber hinaus gesteuert, ob beim Laden der vollständige Cache neu aufgebaut oder lediglich um neue Einträge ergänzt werden soll.

4.2.2. Log-Service

Der Log-Service bildet den Kern der Log-Funktion und ist für das Auslesen und Parsen der Windows-Ereignisprotokoll-Einträge zuständig. Er wird in der Startseite direkt instanziiert, da er mit dynamischen Log-Namen konfiguriert wird, die sich zur Laufzeit ändern können und daher nicht sinnvoll über den Dependency-Injection-Container verwaltet werden können. Konkret werden dem Log-Service beim Erzeugen ein Array der zu lesenden Windows-Ereignisprotokoll-Namen, der Zeitpunkt des letzten IIS-Neustarts als untere Zeitgrenze sowie eine optionale Anfangs-Record-ID je Log-Quelle übergeben. Da der Nutzer in den Einstellungen jederzeit weitere Log-Namen ergänzen oder bestehende entfernen kann, wird bei jeder Änderung dieser Konfiguration eine neue Service-Instanz erzeugt, sodass der interne Cache und die Cursor-Werte konsistent zur aktuellen Auswahl bleiben.

Das Auslesen der Einträge erfolgt über die Methode GetLogsBatch(), welche die EventLogReader-API mit XPath-basierter Filterung direkt auf Betriebssystemebene verwendet: Statt alle Einträge in den Arbeitsspeicher zu laden und dort zu filtern, werden ausschließlich relevante Einträge nach Zeitraum, Schweregrad und Record-ID abgerufen. Das sichert hohe Performance auch bei großen Log-Mengen. Sollte der API-Call fehlschlagen, zum Beispiel wenn die Anwendung ohne Administratorrechte ausgeführt wird oder die Systemkonfiguration des Zielrechners abweicht, greift der Service automatisch auf die ältere EventLog-API als Fallback zurück. Dieses Fallback-Konzept stellt sicher, dass die Log-Funktion auch unter ungünstigen Systembedingungen zuverlässig funktioniert, ohne dass der Nutzer manuell eingreifen muss.

Um eine Doppelverarbeitung bereits bekannter Einträge zu vermeiden, unterstützt der Service ein inkrementelles Ladeverfahren über Record-IDs. Der Service merkt sich nach jedem Ladevorgang die höchste gelesene Record-ID je Log-Quelle; beim nächsten Ladevorgang – etwa durch den Auto-Refresh – werden ausschließlich Einträge mit einer höheren Record-ID abgerufen. Dadurch entfällt das wiederholte Durchsuchen bereits bekannter Einträge, was die Performance beim automatischen Neuladen erheblich verbessert und den Cache auf maximal 5.000 Einträge begrenzt hält. Die statische Methode ParseLog() zerlegt den Rohtext eines Eintrags per Regex in die strukturierten Felder Origin, Metadata und Message.

4.2.3. Anzeige der Logs

Die Startseite übernimmt die gesamte Zustandsverwaltung der Log-Funktion. Nach der Initialisierung werden die Benutzereinstellungen über den Settings-Service geladen und der Log-Service entsprechend konfiguriert. Dabei wird auch der Zeitpunkt des letzten IIS-Neustarts aus der Datenbank gelesen, um die Untergrenze für anzuzeigende Log-Einträge korrekt zu setzen.

Die geladenen Einträge werden in einer MudBlazor-Tabelle dargestellt, die eine Sortierung nach Zeitstempel sowie eine farbliche Kennzeichnung der Log-Level unterstützt. Über eine Suchleiste kann der Benutzer die angezeigten Einträge in Echtzeit durchsuchen, wobei ein Debounce-Mechanismus von 250 Millisekunden unnötige Neuberechnungen bei schneller Eingabe verhindert. Die Suche arbeitet ohne Beachtung der Groß- und Kleinschreibung und durchsucht alle drei strukturierten Felder eines Eintrags gleichzeitig, sodass der Nutzer ohne Vorwissen über das Log-Format gezielt nach Bestandteilen wie einem Klassennamen oder einem Bestellbezeichner suchen kann. Suchtreffer werden in der Tabellenansicht durch farbliche Hervorhebung direkt im Text sichtbar gemacht. Technisch wird der Treffertext dazu in zusätzliche span-Elemente eingebettet, die per CSS-Klasse ausgezeichnet sind, ohne den ursprünglichen Inhalt zu verändern (siehe [10.1.2_Log-Suche](#)

).

Zusätzlich steht ein Auto-Refresh-Mechanismus zur Verfügung, der die Log-Tabelle in einem konfigurierbaren Intervall automatisch aktualisiert. Dabei werden ausschließlich neue Einträge inkrementell nachgeladen, um die Performance zu optimieren. Damit sich überlappende Ladevorgänge bei kurzen Intervallen oder einem manuell ausgelösten Refresh nicht gegenseitig stören, wird jeder laufende Vorgang über ein Sperrflag markiert. Weitere Refresh-Anfragen werden in diesem Fall verworfen, bis der aktuelle Vorgang abgeschlossen ist.

4.3. Implementieren eines IIS-Services

Die IIS-Steuerung wurde als weiteres zentrales Feature der Shop-Toolbox implementiert und ermöglicht das direkte Starten, Stoppen und Neustarten von IIS-Websites ohne den Umweg über den IIS-Manager. Dafür wird die Microsoft.Web.Administration-API über das gleichnamige NuGet-Paket eingebunden.

4.3.1. IIS-Service

Der IIS-Service bildet den Kern der IIS-Steuerung und abstrahiert die Microsoft.Web.Administration-API hinter einer einheitlichen Schnittstelle. Er wird über den Dependency-Injection-Container als Scoped-Dienst bereitgestellt, wodurch bei jeder Blazor-Komponente eine eigene Instanz mit einer neuen IIS-Verbindung erstellt wird. Auf diese Weise wird der jeweils aktuelle Systemzustand korrekt gelesen und es verbleiben keine veralteten Zustände aus einer gecachten Verbindung. Die Verbindung selbst wird intern über ein ServerManager-Objekt gehalten, das beim Erzeugen des Service initialisiert und beim Verwerfen der Komponente wieder freigegeben wird. Dieses Vorgehen vermeidet sowohl das wiederholte Öffnen einer neuen IIS-Verbindung bei jedem Methodenaufruf als auch das längerfristige Halten einer einzigen, gemeinsamen Verbindung über die gesamte Lebensdauer der Anwendung.

Die grundlegenden Steuerungsoperationen Start und Stop sind jeweils in zwei Varianten implementiert: eine parameterlose Variante, die die aktuell in den Einstellungen ausgewählte Site verwendet, sowie eine Kernvariante, die ein konkretes Site-Objekt entgegennimmt und auch von internen Abläufen wie dem Neustart genutzt wird. Alle Operationen sind mit einer Fehlerbehandlung versehen, die dem Nutzer über den bereits vorhandenen Benachrichtigungsdienst farblich differenzierte Toast-Benachrichtigungen mit entsprechenden Erfolgs- oder Fehlermeldungen anzeigt.

4.3.2. Erweiterter Neustart

Die komplexeste Operation stellt der erweiterte Neustart dar. Diese Methode führt mehrere Schritte in einer festgelegten Reihenfolge aus: Zunächst wird die Webseite gestoppt, anschließend werden optional der CSS-Bundler sowie der Asset-Ordner der Site gelöscht, bevor der Application Pool recycelt wird. Nach einer konfigurierbaren Verzögerung von standardmäßig zwei Sekunden, deren Wert über die Einstellungsseite zwischen 0 und 10 Sekunden angepasst werden kann, wird die Site neu gestartet. Diese Wartezeit dient dazu, dem IIS genügend Zeit zu geben, alle laufenden Worker-Prozesse vollständig zu beenden, bevor die neue Instanz startet – ohne diesen Puffer kann es vereinzelt zu Dateisperrungen kommen, die einen erfolgreichen Start verhindern. Abschließend wird der Zeitstempel des Neustarts in der Datenbank gespeichert, damit die Log-Funktion diesen Wert als neue Zeitgrenze verwenden kann. Die optionalen Bereinigungsschritte werden ausschließlich dann ausgeführt, wenn die zu neustartende Site auch tatsächlich als Shopsystem erkannt wird. Bei sonstigen Sites wird der Neustart auf Stop, Application-Pool-Recycle und Start reduziert (siehe [0](#))

Erweiterter Neustart

).

Während des Neustartvorgangs werden die einzelnen Teilschritte bewusst ohne eigene Toast-Benachrichtigungen ausgeführt, um den Benutzer nicht mit mehreren aufeinanderfolgenden Meldungen zu überfluten. Lediglich relevante Operationen wie das Recyceln des Application Pools oder das Löschen des Bundlers geben eine eigene Rückmeldung. Beim Recyceln beendet der IIS kontrolliert den aktuellen Worker-Prozess (w3wp.exe) des Application Pools und startet einen neuen. Dabei werden alle im Prozessspeicher gehaltenen Ressourcen – zwischengespeicherte Anwendungsdaten, geladene Assemblies und offene Dateihandles – freigegeben, während die Konfiguration des Application Pools und laufende Anfragen erhalten bleiben. Im Gegensatz zu Stop/Start ist die Erneuerung dadurch unterbrechungsarm.

Um zu erkennen, ob eine Site ein Shopsystem ist und die shopspezifischen Bereinigungsschritte ausgeführt werden sollen, wurde die Erweiterungsmethode `IsShop()` implementiert. Diese prüft anhand des Dateisystempfads der Site, ob die charakteristischen Verzeichnisse und Dateien eines 4SELLERS-Shopsystems vorhanden sind, nämlich eine `shop.yaml`-Datei sowie ein Themes-Verzeichnis. Die zugehörigen Bereinigungen werden über zwei Erweiterungsmethoden gekapselt: `DeleteShopBundler()` entfernt das vom CSS-Bundler erzeugte Verzeichnis, `DeleteAssetFolder()` leert das Asset-Cache-Verzeichnis der Site. Beide Methoden überspringen nicht vorhandene Verzeichnisse stillschweigend, weil der CSS-Bundler je nach IIS-Konfiguration nicht zwingend bei jedem Start ein eigenes Verzeichnis erzeugt.

4.3.3. Bedienung der IIS-Funktionen

Die Steuerungsbuttons wurden in die bestehende Startseite integriert. Über ein Dropdown-Menü oberhalb der Steuerungsbuttons kann der Nutzer die zu verwaltende IIS-Site auswählen, wobei der aktuelle Status der Site durch einen farblichen Indikator direkt im Dropdown visualisiert wird. Zur schnellen visuellen Orientierung sind die Steuerungsbuttons in Signalfarben gehalten und mit passenden Icons versehen, sodass der Nutzer die gewünschte Aktion auf einen Blick erkennen kann. Ergänzend wurde das Konzept einer „Tray-Icon-Site“ eingeführt: Über die Einstellungen kann der Nutzer eine bestimmte Site als bevorzugte Anwendung auswählen, die anschließend auch über das Symbol der Shop-Toolbox in der Windows-Taskleiste direkt gestartet, gestoppt oder neu gestartet werden kann, ohne das Hauptfenster zu öffnen. Damit wird der typische Anwendungsfall „kurzer Neustart zwischen zwei Code-Änderungen“ mit minimalem Klickaufwand abgedeckt.

Der Neustart-Button ruft eine Wrapper-Methode auf, die nach dem Neustart automatisch den neuen Neustart-Zeitstempel aus der Datenbank lädt und die Log-Tabelle aktualisiert. Dadurch werden die Log-Funktion und die IIS-Steuerung nahtlos miteinander verknüpft: Ein Neustart des Shopsystems führt automatisch dazu, dass in der Log-Tabelle nur noch die Einträge angezeigt werden, die nach dem Neustart entstanden sind.

4.4. Implementieren eines Settings-Services

Der Settings-Service wurde als zentrale Komponente zur benutzerspezifischen Konfiguration der Shop-Toolbox implementiert. Über ihn lassen sich alle relevanten Einstellungen persistent in einer lokalen SQLite-Datenbank speichern und stellt diese allen Komponenten der Anwendung über einen einheitlichen Zugriffspunkt bereit. Als Object-Relational-Mapper kommt dabei Entity Framework Core zum Einsatz, sodass alle Datenzugriffe stark typisiert und ohne manuell formulierte SQL-Anweisungen erfolgen (siehe [10.1.5_Datenbankstruktur](#)

).

Die Wahl von SQLite gegenüber einem zentralen Datenbankserver ergibt sich unmittelbar aus dem Anwendungsfall: Die Shop-Toolbox wird lokal je Entwicklerarbeitsplatz betrieben, benötigt keine Synchronisation zwischen mehreren Nutzern und profitiert von der dateibasierten Speicherung dadurch,

dass keine zusätzliche Server-Infrastruktur erforderlich ist und die Konfiguration einfach gesichert oder zwischen Geräten kopiert werden kann.

4.4.1. Settings-Service

Der Settings-Service kapselt den gesamten Datenbankzugriff und wird über den Dependency-Injection-Container als Scoped-Dienst bereitgestellt. Die Registrierung erfolgt dabei nicht direkt mit der konkreten Klasse, sondern über das Interface `ISettingsService`. Auf diese Weise können die abhängigen Komponenten ausschließlich gegen das Interface programmiert werden, was die spätere Austauschbarkeit der Implementierung und insbesondere das Schreiben von Unit-Tests mit einer Mock-Implementierung deutlich vereinfacht. Beim ersten Erstellen der Instanz lädt die Methode `Load()` die gespeicherten Einstellungen aus der SQLite-Datenbank und hält sie als gecachtes `ToolboxSettings`-Objekt im Arbeitsspeicher. Dadurch wird sichergestellt, dass alle Komponenten stets auf denselben, konsistenten Einstellungsstand zugreifen, ohne bei jedem Zugriff eine Datenbankabfrage auszuführen.

Da EF Core das geladene Objekt automatisch trackt, genügt es, Änderungen direkt am gecachten Objekt vorzunehmen und anschließend `SaveChanges()` aufzurufen, somit ist ein erneutes Laden aus der Datenbank nicht notwendig.

4.4.2. Einstellungen speichern

Für das Speichern von Einstellungsänderungen wurde die Methode `SaveSettingChanges()` implementiert, die ein Lambda als Parameter entgegennimmt. Mit diesem Muster lässt sich jede Einstellung kompakt in einer Zeile speichern aus der Benutzeroberfläche, ohne Code-Duplikation zu erzeugen. Der erste Parameter ist stets ein Lambda der Form `x => x.Property = value`, der zweite Parameter ist der Benachrichtigungsdienst, über den nach jedem erfolgreichen Speichervorgang eine Toast-Benachrichtigung ausgegeben wird. Die Methode kapselt damit drei Aspekte, die andernfalls in jeder Aufrufstelle wiederholt würden: das Anwenden der Änderung auf das gecachte Settings-Objekt, das anschließende Persistieren über EF Core sowie die Rückmeldung an den Nutzer. Tritt beim Speichervorgang ein Fehler auf, beispielsweise weil die Datenbankdatei temporär durch ein Backup-Werkzeug gesperrt ist, wird die Ausnahme abgefangen und stattdessen eine Fehler-Toast-Benachrichtigung ausgelöst, sodass die Oberfläche reaktionsfähig bleibt und der Nutzer den Vorgang gezielt wiederholen kann.

4.4.3. Einstellungsseite

Die Einstellungen werden dem Nutzer über eine dedizierte Einstellungsseite zugänglich gemacht, die in mehrere thematische Unterabschnitte gegliedert ist. Die Log-Einstellungen ermöglichen die Konfiguration des Windows-Ereignisprotokoll-Namens, des Auto-Refresh-Intervalls sowie der Log-Bündelung. Die IIS-Einstellungen steuern die Neustart-Verzögerung sowie die shopspezifischen Bereinigungsoptionen. Ergänzend erlaubt der Bereich „Shops“ die Konfiguration der einzelnen Shop-spezifischen Pfade und Datenbankverbindungen dient. Ein eigener Abschnitt für die VMware-vCenter-Anbindung sowie eine Hilfeseite mit Erläuterungen zu allen Feldern runden die Einstellungsseite ab. Alle Änderungen werden direkt beim Verlassen eines Eingabefeldes gespeichert, sodass kein expliziter Speichern-Button erforderlich ist und der typische Bedienfehler eines Verlassens der Seite ohne Speichern damit konstruktiv ausgeschlossen wird.

4.4.4. ToolboxSettings

Das Datenmodell `ToolboxSettings` folgt einem Singleton-Pattern: Es existiert stets genau ein Datensatz in der Datenbank. Technisch wird dies durch eine Datenbankconstraint (`CK_AppSettings_Singleton`) abgesichert, die auf Datenbankebene erzwingt, dass ausschließlich der Datensatz mit dem Primärschlüssel `Id = 1` gespeichert werden darf. Jeder Versuch, einen zweiten Einstellungsdatensatz

anzulegen, wird damit bereits von der Datenbank abgewiesen, noch bevor die Anwendungslogik eingreift. Das Modell enthält alle benutzerspezifischen Einstellungen der Shop-Toolbox, darunter die Log-Konfiguration, die IIS-Steuerungsparameter sowie allgemeine Pfadeinstellungen. Mehrwertige Felder wie Log-Namen oder Repository-Pfade werden als semikolongetrennte Zeichenketten gespeichert und über dedizierte Hilfsmethoden gelesen und geschrieben, um den Zugriff zu vereinheitlichen und fehleranfällige manuelle String-Operationen zu vermeiden.

Um die Einstellungen über Updates hinaus zu erhalten, wird die Datenbankdatei im AppData-Verzeichnis des jeweiligen Nutzers abgelegt. Beim Anwendungsstart werden ausstehende EF Core Migrationen automatisch angewendet, sodass Erweiterungen des Datenmodells ohne manuellen Eingriff und ohne Datenverlust eingespielt werden.

5. Testen

Die Testphase umfasste manuelle Funktionstests aller implementierten Komponenten. Die einzelnen Features wurden durch gezielte Bedienung der Benutzeroberfläche systematisch auf korrekte Funktionsweise, erwartetes Fehlverhalten und Benutzerfreundlichkeit geprüft.

Bei der Log-Funktion wurde insbesondere verifiziert, dass sämtliche Einträge aus dem Windows-Ereignisprotokoll vollständig und in korrekter chronologischer Reihenfolge dargestellt werden. Darüber hinaus wurden Such- und Filterfunktionen auf ihre Korrektheit geprüft, der Auto-Refresh-Mechanismus auf zuverlässiges inkrementelles Nachladen getestet sowie das Verhalten bei leeren Log-Meldungen überprüft. Auch der automatische Fallback von der EventLogReader-API auf die ältere EventLog-API wurde gezielt getestet, indem die bevorzugte API durch eingeschränkte Berechtigungen zum Fehlschlagen gebracht wurde.

Für die IIS-Steuerung wurden alle drei Grundoperationen Starten, Stoppen und Neustarten systematisch durchgeführt und auf korrekte Statusanzeige sowie entsprechende Toast-Benachrichtigungen geprüft. Der erweiterte Neustart wurde dabei besonders gründlich getestet: Es wurde verifiziert, dass der CSS-Bundler und der Asset-Ordner nach dem Vorgang tatsächlich nicht mehr vorhanden sind und der Shop anschließend in einem sauberen Zustand hochfährt. Ebenso wurde geprüft, dass die shopspezifischen Bereinigungsschritte ausschließlich dann ausgeführt werden, wenn die betreffende Site als Shopssystem erkannt wird.

Der Einstellungsdienst wurde auf persistente Speicherung aller Konfigurationsänderungen getestet. Dabei wurde explizit geprüft, ob die Einstellungen nach einer Neuinstallation der Anwendung vollständig erhalten bleiben, was das zentrale Akzeptanzkriterium dieses Features darstellt. Zusätzlich wurde sichergestellt, dass die automatische Migration des Datenbankschemas beim Update reibungslos funktioniert und keine bestehenden Einstellungswerte überschreibt.

Über das genannte hinaus wurde die Anwendung im Rahmen eines internen Nutzertests ausgewählten Kollegen zur Verfügung gestellt, die die Funktionen unabhängig erprobten und Rückmeldungen zu Bedienbarkeit und Verhalten lieferten. Das Feedback betraf vor allem die Darstellung der Log-Einträge (siehe [5.1.1_Anzeige der Logmeldungen](#)

) sowie die Benutzeroberfläche auf kleineren Bildschirmen (siehe [5.1.2_Verbessern der Ansicht für kleinere Bildschirme](#)

). Die identifizierten Unstimmigkeiten wurden anschließend behoben und erneut getestet.

5.1. Designänderungen

Im Rahmen der Entwicklung wurden mehrere Designänderungen vorgenommen, die auf Basis von Nutzerfeedback und praktischen Erkenntnissen während der Testphase entstanden.

5.1.1. Anzeige der Logmeldungen

Zur Darstellung der Log-Details wurde initial ein modaler Dialog eingesetzt. Da dieser den gesamten Arbeitsbereich blockiert und die Nutzerinteraktion unterbricht, wurde er durch ein Popover ersetzt. Dieses öffnet sich kontextnah neben dem auslösenden Element und erlaubt es, parallel mit der restlichen Oberfläche zu interagieren. Die Entscheidung verbessert die Übersichtlichkeit und reduziert den kognitiven Aufwand beim Wechsel zwischen Log-Ansicht und Hauptinhalt.

5.1.2. Verbessern der Ansicht für kleinere Bildschirme

Die Benutzeroberfläche wurde mithilfe des Bootstrap-Frameworks responsiv gestaltet, sodass eine konsistente Darstellung auf unterschiedlichen Bildschirmgrößen gewährleistet wird. Für eine übersichtliche und flexible Layoutstruktur wurden relative Einheiten sowie das CSS-Flexbox-Modell eingesetzt. Bei kleineren Bildschirmgrößen werden Textbeschriftungen der Steuerungselemente automatisch ausgeblendet und durch kompakte Icon-Darstellungen ersetzt, um die Benutzeroberfläche auch auf eingeschränktem Anzeigeraum übersichtlich zu halten.

6. Fazit und Weiterentwicklung

6.1. Fazit

Das Projektziel, die Shop-Toolbox um eine integrierte Log-Anzeige, eine direkte IIS-Steuerung sowie einen benutzerspezifischen Einstellungsdienst zu erweitern, wurde vollständig erreicht. Alle geplanten Features wurden innerhalb der vorgesehenen Projektzeit von 80 Stunden implementiert, getestet und auf Basis des Nutzerfeedbacks weiter optimiert. Alle definierten Akzeptanzkriterien konnten erfüllt werden: Die Log-Anzeige stellt Einträge vollständig und in korrekter Reihenfolge dar, der erweiterte Neustart gewährleistet einen garantiert sauberen Systemzustand, und die Einstellungen bleiben auch nach Updates und Neuinstallationen vollständig erhalten.

Die Erweiterung vereinfacht den Entwickleralltag spürbar. Der Wechsel zwischen verschiedenen Systemwerkzeugen entfällt, weil Log-Analyse, IIS-Steuerung und Konfiguration nun an einer Stelle liegen. Entwickler können sich dadurch stärker auf ihre eigentlichen Aufgaben konzentrieren, statt sie für administrative Tätigkeiten zu unterbrechen. Auch wirtschaftlich rechnet sich der Aufwand: Die Investition ist nach rund 75 Arbeitstagen amortisiert.

Rückblickend verlief die Projektdurchführung weitgehend planmäßig. Die Abweichung von zwei Stunden weniger in der Analyse- und zwei Stunden mehr in der Dokumentationsphase wurde innerhalb des Gesamtbudgets ausgeglichen. Besonders die frühzeitige Entscheidung, bestehende Technologien der Shop-Toolbox konsequent weiterzuverwenden, erwies sich als richtig — sie reduzierte den Einarbeitungsaufwand und ermöglichte eine nahtlose Integration der neuen Module.

6.2. Weiterentwicklung

Obwohl alle geplanten Projektziele vollständig erreicht wurden, bietet die Shop-Toolbox als wachsendes internes Werkzeug weiteres Potenzial für zukünftige Erweiterungen. Im Folgenden werden mögliche Ansätze skizziert, die den Nutzen der Anwendung für Entwickler weiter steigern könnten.

6.2.1. Verwaltung von Staging VMs

Eine denkbare Erweiterung der Shop-Toolbox wäre die Integration einer VM-Verwaltung für Staging-Umgebungen. Über eine Anbindung an die vSphere REST API könnten Entwickler den Status ihrer Staging-VMs direkt in der Shop-Toolbox einsehen und grundlegende Power-Aktionen wie Starten, Stoppen oder Neustarten ausführen. Zusätzlich wären die Verwaltung von Snapshots sowie das direkte

Herstellen von RDP-Verbindungen denkbar, wodurch der Wechsel zwischen verschiedenen Werkzeugen im Staging-Betrieb weiter reduziert werden könnte.

6.2.2. Verwaltung von Shopsystem Extensions

Darüber hinaus könnte die Shop-Toolbox um eine zentrale Verwaltung von Shopsystem-Extensions erweitert werden. Diese würde es Entwicklern ermöglichen, Extensions direkt aus der Anwendung heraus zu bauen, in die Shopdatenbank zu installieren und deren Versionsstände zu verwalten. Da diese Tätigkeiten aktuell manuell über separate Werkzeuge durchgeführt werden, würde eine Integration in die Shop-Toolbox den Entwicklungsprozess weiter vereinheitlichen und beschleunigen.

6.2.3. Verwaltung von Shopsystem Themes

Als weitere Erweiterungsmöglichkeit bietet sich die Integration einer Theme-Verwaltung an. Über diese könnten Entwickler verfügbare Themes durchsuchen, das aktive Theme des Shopsystems direkt wechseln und die zugehörigen symbolischen Links im Shop-Verzeichnis verwalten. In Kombination mit der bereits implementierten IIS-Steuerung könnte ein Theme-Wechsel automatisch einen Neustart des Shopsystems auslösen, um die Änderungen unmittelbar wirksam werden zu lassen.

7. Glossar

Begriff	Erklärung
Blazor / Blazor Desktop	Ein Framework von Microsoft, das die Entwicklung von Web- und Desktop-Anwendungen mit C# anstelle von JavaScript ermöglicht. In diesem Projekt wird Blazor in einer Desktop-Anwendung eingesetzt, die in einem WPF-Fenster mit WebView2 läuft.
Bootstrap	Ein weit verbreitetes CSS-Framework, das fertige Layoutklassen und Gestaltungsregeln bereitstellt. Es erleichtert die responsive Gestaltung von Benutzeroberflächen für unterschiedliche Bildschirmgrößen.
CSS-Bundler / CSS-Cache	Der CSS-Bundler fasst mehrere Stylesheet-Dateien zu einer einzigen Datei zusammen, um die Ladezeiten zu optimieren. Der CSS-Cache speichert diese gebündelte Datei, damit sie nicht bei jedem Aufruf neu erzeugt werden muss.
Debounce-Mechanismus	Eine Technik in der Softwareentwicklung, die verhindert, dass eine Funktion bei jeder Tastatureingabe sofort ausgeführt wird. Stattdessen wird gewartet, bis der Nutzer eine kurze Pause macht (hier 250 ms), bevor die eigentliche Aktion ausgelöst wird.
Dependency Injection (DI)	Ein Entwurfsmuster, bei dem Abhängigkeiten einer Klasse von außen bereitgestellt werden, anstatt sie selbst zu erzeugen. Der DI-Container verwaltet die Erstellung und Lebensdauer dieser Objekte automatisch.
Entity Framework Core (EF Core)	Ein objektrelationaler Mapper von Microsoft für .NET, der den Datenbankzugriff vereinfacht. Entwickler arbeiten mit C#-Objekten statt SQL-Abfragen; EF Core übersetzt diese Operationen automatisch in Datenbankbefehle.
EventLog-API / EventLogReader-API	Zwei .NET-Programmierschnittstellen zum Auslesen des Windows-Ereignisprotokolls. Die neuere EventLogReader-API unterstützt XPath-basierte Filterung direkt auf Systemebene und ist damit deutlich effizienter.
Git / Git-Repository	Git ist ein weit verbreitetes Versionsverwaltungssystem für Quellcode. Ein Repository ist der zentrale Speicherort, in dem alle Versionen und Änderungen des Codes nachvollziehbar gespeichert werden.
IIS (Internet Information Services)	Der Webserver von Microsoft für Windows-Systeme, der Webanwendungen bereitstellt und verwaltet. Das 4SELLERS Shopsystem wird über den IIS gehostet und kann über den IIS-Manager oder per API gesteuert werden.
Inkrementelles Laden	Eine Technik, bei der nur neue oder geänderte Daten nachgeladen werden, anstatt alle Daten neu abzurufen. Im Projekt werden nur Log-Einträge mit einer höheren Record-ID als beim letzten Abruf geladen.
JetBrains Rider	Eine integrierte Entwicklungsumgebung (IDE) von JetBrains, die speziell für .NET- und C#-Projekte optimiert ist. Sie bietet Funktionen wie Code-Vervollständigung, Debugging und Refactoring.
Medienbruch	Ein Wechsel zwischen verschiedenen Anwendungen während eines Arbeitsprozesses, der den Arbeitsfluss unterbricht. Beispielsweise das Wechseln zwischen Shop-Toolbox, IIS-Manager und Windows-Ereignisanzeige.

Migration (EF Core)	Eine versionierte Änderungsbeschreibung des Datenbankschemas, die von EF Core automatisch angewendet wird. Migrationen stellen sicher, dass Datenbankstruktur und Anwendungscode nach einem Update synchron sind.
MudBlazor	Eine Komponentenbibliothek für Blazor, die fertige UI-Elemente wie Tabellen und Schaltflächen nach dem Material-Design-Standard bereitstellt. Sie gewährleistet ein einheitliches Erscheinungsbild in der Shop-Toolbox.
.NET / .NET 10	Eine plattformübergreifende Laufzeit- und Entwicklungsplattform von Microsoft. .NET 10 ist die Version, auf der das 4SELLERS Shopsystem und die Shop-Toolbox basieren.
NuGet-Paket	Ein Paketformat für den .NET-Paketmanager NuGet, über den wiederverwendbare Bibliotheken in .NET-Projekte eingebunden werden können. Im Projekt wird z. B. das Microsoft.Web.Administration-Paket genutzt.
Popover	Ein UI-Element, das sich kontextnah neben einem auslösenden Element öffnet, ohne den restlichen Inhalt zu blockieren. Im Gegensatz zu einem modalen Dialog bleibt die übrige Benutzeroberfläche weiterhin bedienbar.
Record-ID	Eine eindeutige, fortlaufend vergebene Nummer für jeden Eintrag im Windows-Ereignisprotokoll. Sie ermöglicht das inkrementelle Nachladen, da neue Einträge stets eine höhere Record-ID als bereits geladene besitzen.
Regex (Regulärer Ausdruck)	Eine Zeichenkette, die ein Suchmuster beschreibt und zum Finden oder Verarbeiten von Textinhalten verwendet wird. Im Log-Service wird Regex genutzt, um den Rohtext von Log-Einträgen in strukturierte Felder zu zerlegen.
Responsive Design	Ein Gestaltungsansatz, bei dem sich das Layout automatisch an unterschiedliche Bildschirmgrößen anpasst. Dies wird im Projekt mithilfe von Bootstrap und CSS-Flexbox umgesetzt.
Scoped-Dienst	Ein Dienst im DI-System, der für jeden Anfragekontext neu instanziiert wird. Dadurch erhält jede Blazor-Komponente eine eigene, frische Instanz mit aktuellem Systemzustand.
Singleton-Pattern	Ein Entwurfsmuster, das sicherstellt, dass genau eine Instanz eines Datensatzes existiert. Im Projekt wird dies durch eine Datenbankconstraint erzwungen, die nur den Einstellungsdatensatz mit Id = 1 erlaubt.
SQLite	Eine leichtgewichtige, dateibasierte Datenbank-Engine ohne separate Serverinstallation. Die gesamte Datenbank wird in einer einzigen Datei gespeichert, was sie ideal für lokale Anwendungseinstellungen macht.
Toast-Benachrichtigung	Eine kurze, nicht-blockierende Statusmeldung, die für eine begrenzte Zeit am Bildschirmrand eingeblendet wird. Im Projekt werden Toast-Benachrichtigungen farblich differenziert, um Erfolgs- oder Fehlermeldungen anzuzeigen.
vSphere REST API	Eine Programmierschnittstelle der VMware-Virtualisierungsplattform vSphere, über die virtuelle Maschinen per Code gesteuert werden können. Sie wird im Ausblick als mögliche Erweiterung für Staging-VM-Verwaltung erwähnt.
WebView2	Eine Microsoft-Komponente, die das Chromium-basierte Rendering-Modul in Desktop-Anwendungen einbettet. In der Shop-Toolbox

	ermöglicht es, Blazor-Komponenten innerhalb eines klassischen WPF-Fensters darzustellen.
Windows-Ereignisanzeige / -protokoll	Ein integriertes Windows-Werkzeug, das Systemmeldungen, Fehler und Sicherheitsereignisse in Protokollen speichert. Im Projekt wird es als Quelle für die Log-Analyse des Shopsystems genutzt.
WPF (Windows Presentation Foundation)	Ein UI-Framework von Microsoft für Desktop-Anwendungen unter Windows. In der Shop-Toolbox dient WPF als Host-Fenster, in dem die Blazor-Weboberfläche über WebView2 eingebettet wird.
XPath	Eine Abfragesprache zur Navigation und Filterung in XML-Dokumenten. Die EventLogReader-API nutzt XPath, um Log-Einträge direkt auf Systemebene zu filtern, ohne alle Einträge laden zu müssen.

8. Quellen

Frameworks & Plattformen

Microsoft: Blazor documentation – <https://learn.microsoft.com/en-us/aspnet/core/blazor/>

Microsoft: .NET documentation – <https://learn.microsoft.com/en-us/dotnet/>

Microsoft: Entity Framework Core – <https://learn.microsoft.com/en-us/ef/core/>

Microsoft: WebView2 documentation – <https://learn.microsoft.com/en-us/microsoft-edge/webview2/>

IIS & Windows

Microsoft: Internet Information Services (IIS) – <https://learn.microsoft.com/en-us/iis/>

Microsoft: Windows Event Log (EventLogReader) – <https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.eventing.reader.eventlogreader>

Microsoft: EventLog Class – <https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.eventlog>

Bibliotheken & Tools

MudBlazor: MudBlazor documentation – <https://mudblazor.com/docs/overview>

SQLite: SQLite documentation – <https://www.sqlite.org/docs.html>

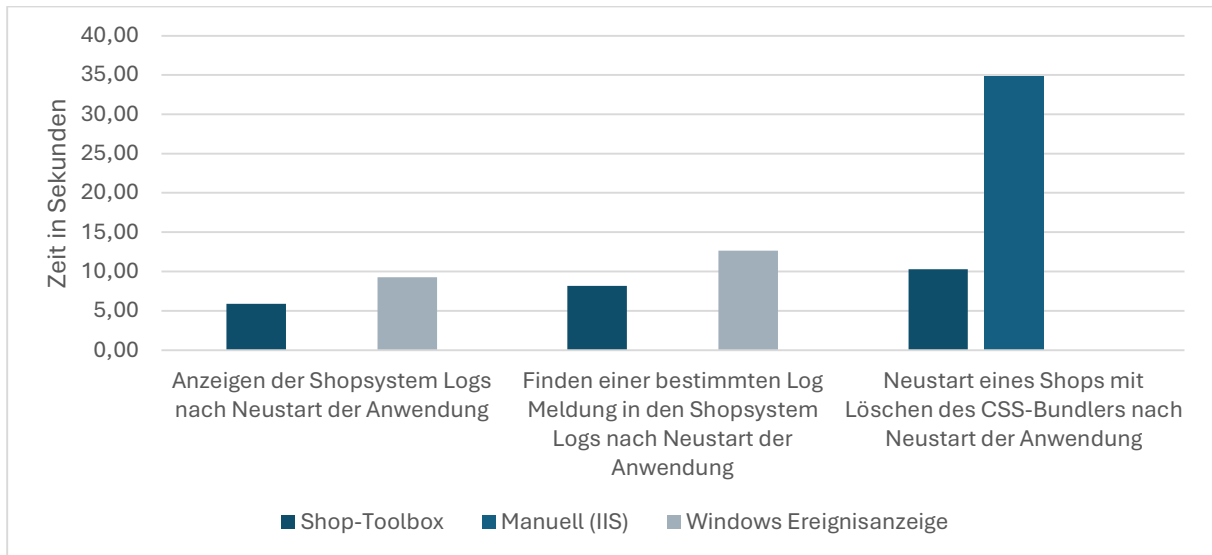
Bootstrap: Bootstrap documentation – <https://getbootstrap.com/docs/>

NuGet: Microsoft.Web.Administration – <https://www.nuget.org/packages/Microsoft.Web.Administration>

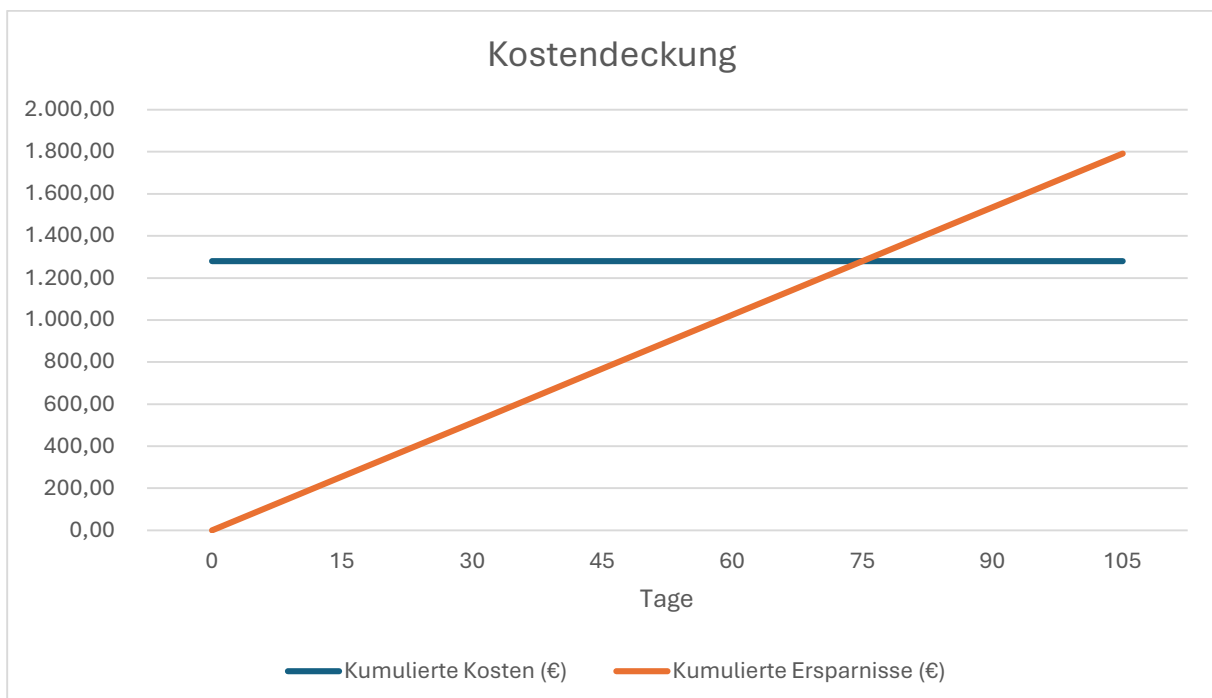
9. Anhänge

9.1. Aufwand und Nutzen

9.1.1. Bearbeitungszeiten im Vorher-Nachher-Vergleich (in Sekunden)



9.1.2. Kostendeckung



10. Projektumsetzung

10.1.1. Startseite

The screenshot displays the 4Sellers Toolbox application interface. On the left, a sidebar contains navigation links: Home, Themes, Extensions, Shop, IIS-Seiten, and Einstellungen. The main area is divided into two sections. The top section, titled 'IIS App', shows the configuration for 'Shopsystem-V2-T4-26082025'. It includes three buttons: 'START' (green), 'NEUSTART' (orange), and 'STOP' (red). Below these, there are two sections for extensions: 'Geladene Extensions seit letztem Neustart' (loaded) and 'Nicht geladene Extensions seit letztem Neustart' (not loaded). The 'Geladene Extensions' section lists 'Toolbox.CrossSelling' and 'Toolbox.Captcha'. The 'Nicht geladene Extensions' section lists 'Toolbox.Captcha'. The bottom section, titled 'Suche', displays a log of errors. The log table has columns for 'Zeit', 'Ebene', and 'Nachricht'. The errors are categorized by 'Information' and 'Error'. The 'Error' messages indicate issues with translation detection and view component rendering.

Zeit	Ebene	Nachricht
24.11.2025 11:20:53	Information	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:20:53.610 Message: [Microsoft.Hosting.Lifetime] Application is shutting down...
24.11.2025 11:20:53	Information	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:20:53.611 Message: [Microsoft.Hosting.Lifetime] Application is shutting down...
24.11.2025 11:00:06	Error	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:00:06.539 Message: [ForSellers.Redwood.Web.Domain.Localization.MissingTranslationDetection] Failed to create translations for Fachhandel für BKT, Michelin, Alliance, Kenda und Vredestein Traktorreifen / Schlepperreifen & Shell Schmierstoffe Additional information: * [Error] Translation_Key_StringLength
24.11.2025 11:00:06	Error	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:00:06.539 Message: [ForSellers.Redwood.Web.Domain.Localization.MissingTranslationDetection] Failed to create translations for Fachhandel für BKT, Michelin, Alliance, Kenda und Vredestein Traktorreifen / Schlepperreifen & Shell Schmierstoffe Additional information: * [Error] Translation_Key_StringLength
24.11.2025 11:00:06	Error	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:00:06.540 Message: [ForSellers.Redwood.Web.Domain.Localization.MissingTranslationDetection] Failed to create translations for Shell Schmierstoffe, Michelin, BKT, Shell Motorenöl, Kabat, Landwirtschaftsreifen, Traktorreifen, Treckerreifen, Schlepperreifen Additional information: * [Error] Translation_Key_StringLength
24.11.2025 11:00:06	Error	Origin: ShopId: 1 Metadata: log4net.HostName: WS-119 ShopId: 1 Timestamp: 2025.11.24 11:00:06.540 Message: [ForSellers.Redwood.Web.Domain.Localization.MissingTranslationDetection] Failed to create translations for Shell Schmierstoffe, Michelin, BKT, Shell Motorenöl, Kabat, Landwirtschaftsreifen, Traktorreifen, Treckerreifen, Schlepperreifen Additional information: * [Error] Translation_Key_StringLength
24.11.2025 11:00:05	Error	Origin: ShopId: 1 Metadata: ActionId: a4eb4610-f529-4fec-b738-2603a9d3ddb ViewComponentId: 201fe2ca-0917-442a-a925-f9f9dc10d987 ShopId: 1 IsGuest: True RequestPath: / log4net.HostName: WS-119 CustomerId: -2147483648 ViewComponentName: ForSellers.Redwood.Web.Shop.ViewComponents.WidgetViewComponent RequestId: 40000059-0005-fe00-b63f-84710c7967bb ActionName: ForSellers.Redwood.Web.Shop.Controllers.HomeController.Index (4sellers.Redwood.Web.Shop) Timestamp: 2025.11.24 11:00:05.385 Message: [WidgetViewComponent.InvokeAsync] Error at invoking widget viewcomponent ExternalBasketButton. Exception~1: A view component named 'ExternalBasketButton' could not be found. A view component must be a public non-abstract class, not contain any generic parameters, and either be decorated with 'ViewComponentAttribute' or have a class name ending with the 'ViewComponent' suffix. A view component must not be decorated with 'NonViewComponentAttribute' at Microsoft.AspNetCore.Mvc.ViewComponents.DefaultViewComponentHelper.InvokeAsync(String name, Object arguments) at ForSellers.Redwood.Web.Shop.ViewComponents.WidgetViewComponent.InvokeAsync(String location, Object additionalData) in C:\Dev_Git\Standard\Shopsystem-V2-V4-28102025\Shop\Shop\ViewComponents\WidgetViewComponent.cs:line 36

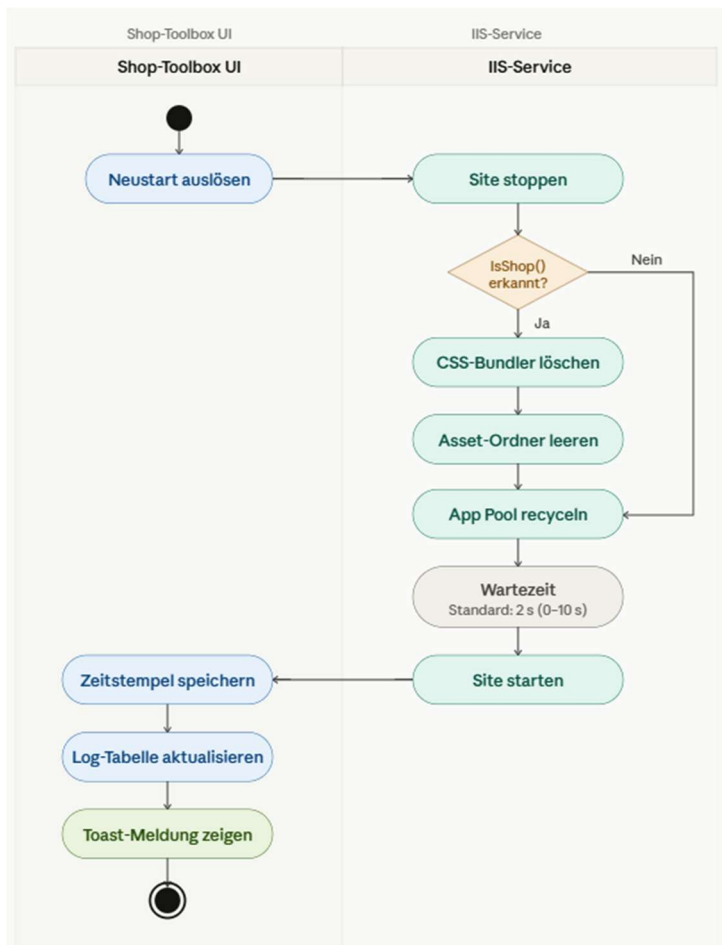
Logs pro Seite 50 1-50 von 693

10.1.2. Log-Suche

The screenshot displays the IIS App management interface for 'Shopsystem-V2-T4-26-01-2026'. On the left, a sidebar contains navigation icons and a 'Toolbox' section with buttons for 'START', 'NEUSTART', and 'STOP'. Below these are sections for 'Geladene Extensions seit letztem Neustart' and 'Nicht geladene Extensions seit letztem Neustart'. The main area is titled 'Suche' and shows a list of log entries for the 'Application' component. The log entries are organized into columns: 'Zeit' (Time), 'Ebene' (Level), and 'Nachricht' (Message). The messages include information about the application shutting down, starting, and loading various components like caches, settings, and object extensions. The interface also includes a search filter icon, a refresh icon, and a pagination bar at the bottom right showing 'Logs pro Seite 50' and '1-50 von 152'.

Zeit	Ebene	Nachricht
27.03.2026 13:48:42	Information	Message: [Microsoft.Hosting.Lifetime] Application is shutting down...
27.03.2026 13:27:47	Information	Message: [Microsoft.Hosting.Lifetime] Application started. Press Ctrl+C to shut down.
27.03.2026 13:27:43	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Warming up caches
27.03.2026 13:27:43	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading cache settings
27.03.2026 13:27:43	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading settings
27.03.2026 13:27:43	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading object extensions
12.03.2026 08:11:00	Information	Message: [Microsoft.Hosting.Lifetime] Application is shutting down...
12.03.2026 07:50:02	Information	Message: [Microsoft.Hosting.Lifetime] Application started. Press Ctrl+C to shut down.
12.03.2026 07:50:01	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Warming up caches
12.03.2026 07:50:01	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading cache settings
12.03.2026 07:50:00	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading settings
12.03.2026 07:50:00	Information	Message: [ForSellers.Redwood.Web.Core.Application.Web Application Initializer] Loading object extensions

10.1.3. Erweiterter Neustart



10.1.4. Log-Popover



10.1.5. Datenbankstruktur

Settings			
int	Id	PK	Singleton: Id = 1
string	IisAppName		nullable
string	LogName		nullable
string	ThemeRepositoryPath		
string	ExtensionsRepositoryPath		
string	GeneralFolderPath		
int	AutoRefreshSeconds		
bool	AutoRefreshEnabled		
bool	OnlySinceRestart		
bool	BundleLogs		
bool	DeleteBundlerOnShopRestart		
bool	DeleteAssetsOnShopRestart		
bool	RestartShopOnThemeChange		
int	RestartDelaySeconds		
long	TrayIconIisSite		
string	PinnedThemeGroups		nullable
string	PinnedExtensionGroups		nullable



ShopSettings			
int	Id	PK	
long	SiteId		IIS Site ID
string	ShopYamlPath		
string	ThemeFolderPath		
int	ToolboxSettingsId	FK	

AppInfo			
int	Id	PK	Singleton: Id = 1
datetime	StartTime		
datetime	IisRestartTime		

ShopDatabaseConnections		
int	Id	PK
string	Server	
string	Database	
string	User	
string	Password	
int	MaxPoolSize	
bool	Encrypt	
bool	TrustServerCertificate	